PRÁCTICA 6 – "Detección de Bordes"

Objetivo: implementación y aplicación de distintos detectores de borde.

PRIMERA PARTE: Generación de imágenes. Análisis de perfiles de intensidad.

1. A partir de la lectura de la imagen "*img.png*", genera las siguientes imágenes:
   - Imagen de intensidad `I`.
   - Imagen de intensidad suavizada `Igauss`: imagen resultado de aplicar a `I` un filtro gaussiano 5x5 y desviación típica 1.4.
2. En dos ventanas, una para cada una de las dos imágenes anteriores, representa:
   - En una gráfica superior: la imagen de intensidad con las siguientes líneas horizontales representadas sobre la imagen, cada una de un color y dadas por las filas: *int(0.25xN)*, *int(0.5xN)* y *int(0.75xN)*, siendo *N* el número de filas de la imagen.
   - En una gráfica inferior: los perfiles de intensidad correspondientes a las líneas horizontales anteriores, representados en los mismos colores y con una leyenda que indique el valor de la fila a la que se corresponde el perfil.

   Observación:
   - Los perfiles deben representarse en la misma escala: eje X: desde 0 hasta el número de columnas de la imagen; eje Y: desde 0 hasta 270.
   - Utiliza *plt.axhline* para representar una determinada línea sobre una imagen.
3. Analiza las representaciones y extrae conclusiones en relación a la caracterización de los píxeles de borde y los fundamentos metodológicos de los algoritmos empleados en su detección.

SEGUNDA PARTE: Implementación de detector de bordes basado en primera derivada.

4. Implementa la siguiente función (almacenada en fichero Python *funcion_calcular_gradiente.py*) para calcular los gradientes de una imagen requeridos para aplicar un detector de bordes basado en primera derivada:

```
Gx, Gy, ModG = calcular_gradiente (I, Hx, Hy)
```

   Esta función deberá:

   - Para cada píxel de la imagen de entrada `I`, obtener las componentes horizontal y vertical, `Gx y Gy`, del vector gradiente calculadas considerando las máscaras de convolución `Hx y Hy` respectivamente y considerando *padding* '*replicate'*.
   - Obtener la magnitud del vector gradiente `ModG`.

5. Aplica la función anterior con las máscaras de convolución de Sobel a la imagen de intensidad `I` y a la imagen de intensidad suavizada `Igauss` y visualiza como imágenes la siguiente información:
   - Magnitud de `Gx`
   - Magnitud de `Gy`
   - Magnitud `ModG` del vector gradiente.

   Analiza los resultados.

   Observaciones y organización de las gráficas:

   - Se debe aplicar `subplot` para generar las 6 representaciones en una misma ventana (columna de la izquierda para las representaciones correspondientes a `I`, y columna de la derecha para las representaciones correspondientes a `Igauss`).
   - Todas las gráficas deben utilizar la misma equivalencia para asociar el negro (nivel de intensidad 0) y blanco (nivel de intensidad 255) con los valores representados. En todas las representaciones, el negro (0) y el blanco (valor 255) de las imágenes, se deben corresponder con el valor mínimo y el máximo de `ModG` de `I`, respectivamente.

   Ayuda de programación: se facilita la función *reescalar_valores_matriz_a_imagen*

6. Utilizando como umbrales el 10, 25, 50 y 75% del valor máximo de la matriz magnitud del vector gradiente obtenida para la imagen de intensidad `I`, representar las imágenes binarias de bordes de `I` e `Igauss` resultado de la aplicación del detector de bordes de Sobel.

   Organización de las gráficas:

   Se debe utilizar la misma estructura del apartado anterior para representar las imágenes binarias de `I` e `Igauss` con los umbrales establecidos (columna de la izquierda para mostrar las binarizaciones de `I`, columna de la derecha para las de `Igauss`).

7. **Adelgazamiento de bordes.**

   Utilizando la función facilitada

   ```
   MagGradSnM = aplicar_supresion_no_maxima(Gx, Gy, MagGrad)
   ```

   a) Aplica el algoritmo de supresión no máxima a la magnitud del gradiente de la imagen suavizada, `Igauss`.

   b) Visualiza como imágenes las magnitudes del gradiente de entrada y salida del algoritmo (1 ventana, dos imágenes distribuidas en 2 filas y 1 columna).

   c) Utilizando como umbrales el 10, 25, 50 y 75% del valor máximo de la matriz magnitud del vector gradiente obtenida para la imagen de intensidad suavizada `Igauss`, binariza ambas magnitudes del gradiente y muestra los resultados como imágenes (1 ventana, 4 filas y 2 columnas de imágenes; las filas se corresponden con los umbrales y las columnas con el tipo de magnitud de gradiente que se está binarizando).

Objetivos teóricos:

- Fundamentos de las etapas metodológicas de un detector de bordes basados en primera derivada. Análisis de la información de la magnitud del vector gradiente y de sus componentes y su posterior binarización.
- Fundamentos de algoritmo de adelgazamiento de bordes basado en supresión no máxima y su integración en un detector de bordes basado en primera derivada.

TERCERA PARTE: Detección de bordes con OpenCV

8. Detector de bordes basados en primera derivada: Sobel

a) Aplica las funciones de OpenCV *Sobel* y *magnitude* para calcular las componentes del vector gradiente `Gx`, `Gy` y su magnitud de la imagen `I`.

b) Genera una imagen de bordes mediante el umbral obtenido mediante el método de Otsu:

- Utiliza la función *cv2.threshold* para obtener una imagen de bordes utilizando como umbral el obtenido mediante el método de Otsu.

- Para aplicar el método de Otsu en la binarización, la función *cv2.threshold* debe recibir como entrada la magnitud del gradiente en formato *uint8* con sus valores escalados a 0-255 (utiliza para ello la función *normalize* con la opción *cv2.CV_8U*) y configurarla para que aplique el umbral que genera el método de Otsu (*cv2.THRESH_BINARY* + *cv2.THRESH_OTSU*).

  Observación: la opción *cv2.CV_8U* en *normalize* permite redondear los valores al entero más cercano. La opción de utilizar *astype(np.uint8)* a la salida de *normalize* (sin *cv2.CV_8U*) trunca los valores decimales y siempre ajuste al entero más bajo.

- La función devuelve el umbral utilizado y la imagen de bordes en *uint8*. Llamemos a este valor de umbral *umbralOtsu*.

c) Utilizando la función *threshold*, genera una imagen de bordes aplicando un umbral de valor 0.4**umbralOtsu.*

d) Visualiza las dos imágenes de bordes obtenidas (1 ventana, 2 imágenes distribuidas en 2 filas y 1 columna, en las que aparezca en el título el valor del umbral utilizado).

Objetivos teóricos:

- Implementación de detector de bordes representativo basado en primera derivada en OpenCV.
- Aplicación y comprensión de todos los parámetros de entrada y salida de la función *threshold* para la etapa final de binarización: uso de umbral manual y generación de umbral de forma automática mediante el método de Otsu.

9. Detector de bordes basados en primera derivada: Canny

a) Utilizando la función de OpenCV *Canny*, aplica sobre la imagen de intensidad `I` un detector de bordes *Canny* con umbrales superior e inferior: *umbralMayor = 100* y *umbralMenor = 0.4*umbralMayor*. Aplica nuevamente el detector con la pareja de umbrales *umbralMayor = 700* y *umbralMenor = 0.4*umbralMayor*.

b) Visualiza las dos imágenes de bordes obtenidas (1 ventana, 2 imágenes distribuidas en 2 filas y 1 columna, en las que aparezca en el título el valor del umbral utilizado) y justifica los resultados.

c) Teniendo en cuenta las siguientes observaciones, resume los distintos pasos procedimentales (interpretando las entradas-salidas de cada paso) que aplica el detector de bordes de *Canny* para, a partir de una imagen de intensidad, obtener su imagen binaria de bordes.

Observaciones:

- El detector de Canny en OpenCV aplica internamente un suavizado Gaussiano 5×5 con σ = 1.4. En el artículo de John Canny (1986) se define que el suavizado previo debe hacerse con una Gaussiana de desviación estándar σ, y se sugiere: "A good practical value for σ is between 1.0 and 2.0; I found σ = 1.4 works well for most images."
- OpenCV usa el operador Sobel para calcular las componentes del vector de gradiente.

10. Postprocesado de imagen binaria de bordes de Canny. Aplica los siguientes pasos para generar una detección de bordes de la carretera:

a) Aplica sobre la imagen de intensidad `I` un detector de bordes *Canny* con umbrales superior e inferior *umbralMayor = 700* y *umbralMenor = 0.4*umbralMayor*.

b) Utilizando *cv2.morphologyEx* con una vecindad definida por una única fila de 7 píxeles, aplica una apertura morfológica sobre la imagen de bordes resultado de *Canny*.

c) Aplica un filtro de máximos de vecindad definida por una caja rectangular de 7 filas y 41 columnas al resultado de la apertura.

d) Utilizando las funciones de *OpenCV bitwise* (distintas funciones que operan sobre imágenes binarias *uint8* de valores 0-255, para realizar operaciones lógicas bit a bit), realiza la operación *and* de la imagen binaria de bordes de *Canny* y la negación lógica del resultado del filtrado de máximos del apartado anterior.

e) Visualiza las 4 imágenes binarias que ilustran el postprocesado aplicado.

f) En un directorio de Imágenes Generadas, almacena la imagen binaria resultado del postprocesamiento *deteccionBordesCarretera.png* (utiliza la función *cv2.imwrite* con la opción *[cv2.IMWRITE_PNG_COMPRESSION, 0]* para no aplicar compresión.

Objetivos teóricos:

- Conocer los fundamentos y etapas del detector de bordes de Canny.
- Implementación del detector de Canny en OpenCV.
- Aplicación de técnicas morfológicas de postprocesado de imágenes binarias para la segmentación de contornos de interés.

11. Detector de bordes basados en segunda derivada: Laplaciana (sin y con suavizado gaussiano) y Laplaciana de la Gaussiana

Detector de bordes *Laplaciana*

a) Utilizando la función de OpenCV *Laplacian* con *ksize* = 1 (esta opción utiliza como *kernel* de convolución la máscara *Laplaciana* de 4-vecinos) y configuración para trabajar en *float32*, calcula la *Laplaciana* de la imagen de entrada `I`.

b) Aplica el detector de cruces por cero facilitado en el fichero de funciones *funciones_deteccion_bordes_segunda_derivada.py* con tres umbrales: *umbral1* = umbral dado por defecto por la función; *umbral2=5*umbral1*; *umbral3= 10*umbral1*.

c) Visualiza las tres imágenes de bordes en una misma ventana (3 filas y 1 columna de imágenes) e interpreta el significado del umbral.

d) Repite los apartados anteriores, a), b) y c), aplicando el proceso a la imagen suavizada mediante un *Filtro Gaussiano* de *sigma = 1* y *W = 6*sigma + 1*.

Detector de bordes *Laplaciana de la Gaussiana* (*LoG*)

e) Aplica el detector *Laplaciana de la Gaussiana* a la imagen `I` con los 3 umbrales calculados con el mismo criterio que el utilizado en el apartado anterior. Para generar el *kernel* de convolución utiliza la función facilitada *generar_kernel_log* con sigma=1.

f) Visualiza las tres imágenes de borde en una misma ventana (3 filas y 1 columna de imágenes).

Análisis de resultados

g) Analiza las tres ventanas gráficas de resultados: *Laplaciana* sin suavizar, *Laplaciana* suavizando y *Laplaciana de la Gaussiana*.

Objetivos teóricos:

- Implementación de detectores de bordes basados en segunda derivada en OpenCV.
- Conocer los fundamentos y etapas del detector de bordes Laplaciana y Laplaciana de la Gaussiana.